

Entrada, saída e Processamento de dados

Vídeo sugerido:

<https://youtu.be/cBUc5nvUlsE>

Processamento de dados

Vídeo sugerido:

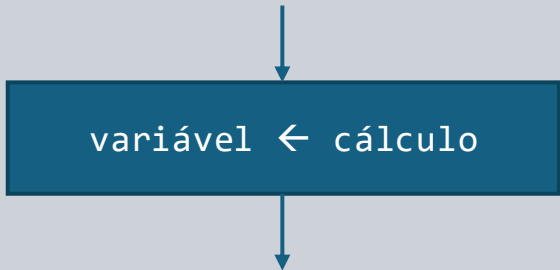
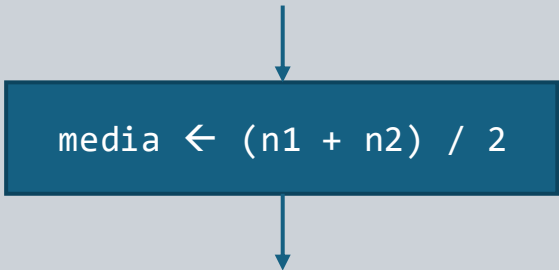
<https://youtu.be/MSErL5fh7hg>

Processamento de dados

O **processamento de dados** é uma etapa fundamental na lógica de programação, na qual instruções são executadas para **manipular informações**.

Ele é utilizado sempre que precisamos realizar operações, como **cálculos** ou transformações de dados.

Segue abaixo a definição Processamento de dados no Fluxograma e no Python:

	Sintaxe	Exemplo
Fluxograma	 <pre>variável ← cálculo</pre>	 <pre>media ← (n1 + n2) / 2</pre>
Python	<pre>variável = cálculo</pre>	<pre>media = (n1 + n2) / 2</pre>

Que tal descobrir os resultados codificando?

Em um arquivo .py coloque as variáveis com os valores:

```
Code 1 a = 10
      2 b = 2
      3 c = 3
      4 x = 6.5
      5 y = 2.7
      6 z = 4.0
```

Na variável **resp** atribua o cálculo a ser conferido

```
Code 7 resp = a + b * c
```

Em um print coloque um texto com a expressão apresentando a variável **resp** no final

```
Code 8 print("1. resp = a + b * c -> ", resp)
```

Execute o código e confira a resposta:

Output

```
1. resp = a + b * c ->
```

16

Resposta

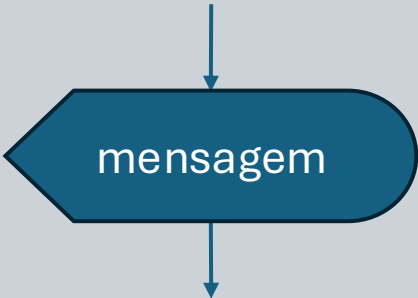
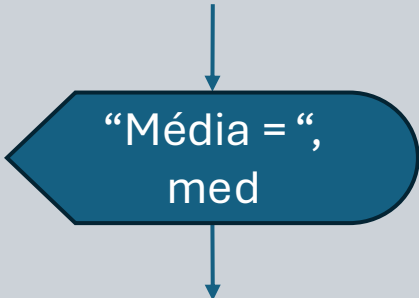
Saída de dados



Saída de dados

A **saída de dados** é uma das instruções mais importantes para o usuário, pois é responsável por **exibir** as informações desejadas. Ela representa a interação do sistema com o usuário e é realizada por meio de dispositivos de saída, como o monitor ou a impressora.

Segue abaixo a definição de saída de dados no Fluxograma e no Python:

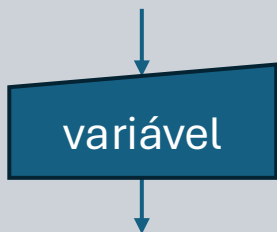
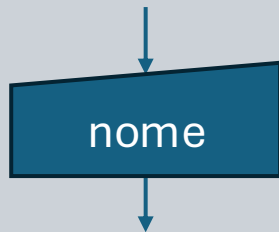
	Sintaxe	Exemplo
Fluxograma		
Python	<code>print(mensagem)</code>	<code>print("Media = ", m)</code>

input()
Entrada de dados

Entrada de dados

A **entrada de dados** é a instrução responsável por **capturar** valores, geralmente **digitados** pelo usuário, e armazená-los em uma variável na memória. Ela representa a interação do usuário com o sistema e é realizada por meio de dispositivos de entrada, como o teclado ou o mouse.

Segue abaixo a definição de entrada de dados no Fluxograma e no Python:

	Sintaxe	Exemplo
<i>Fluxograma</i>		
<i>Python</i>	<code>variável = input(mensagem)</code>	<code>nome = input("Nome: ")</code>

Para utilizarmos a entrada de dados devemos nos preocupar com algo:

O INPUT() LÊ SOMENTE DADOS STR()

Por este motivo precisamos usar o casting()

```
1  # Fazendo o casting() FORA do input
2  nome = input("Nome: ")
3  idade = input("idade: ")
4  idade = int(idade)
5  altura = input("Altura: ")
6  altura = float(altura)
7
8  # Fazendo o casting() NO do input
9  nome = input("Nome: ")
10 idade = int(input("idade: "))
11 altura = float(input("Altura: "))
```

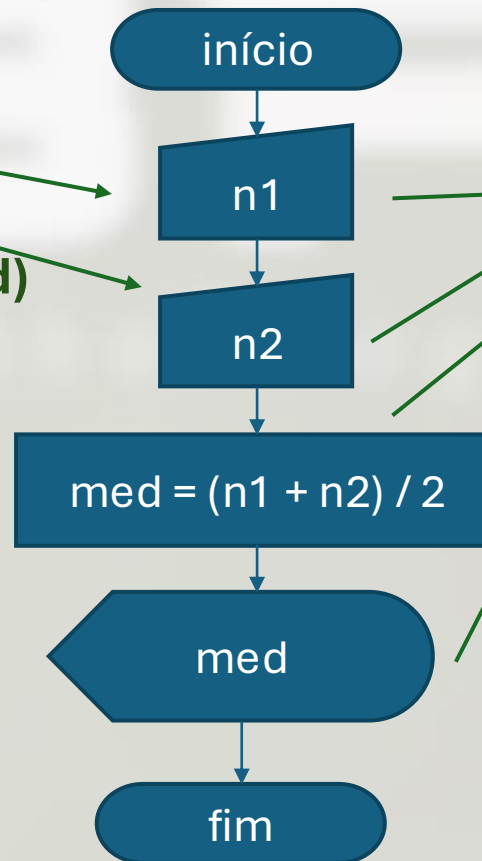
Construção de um Algoritmo completo

No exemplo abaixo consta a construção completa de um algoritmo:

Narrativa, Fluxograma e código em Python

Problema: Fazer um algoritmo que pegue dois valores do usuário e calcule a média

FLUXOGRAMA:



NARRATIVA:

- Digitar o primeiro numero (**ed**)
- Digitar o segundo numero (**ed**)
- Calcular a média dos 2 números (**pd**)
- Exibir o resultado (**sd**)

PYTHON: Código

```
1 n1 = float(input("Digite o primeiro número: "))
2 n2 = float(input("Digite o segundo número: "))
3 med = (n1 + n2) / 2
4 print(f"Média = {med}")
```

TELA: Execução do algoritmo

```
Digite o primeiro número: 4
Digite o segundo número: 9
Média = 6.5
```

Formatações do print()

Videos afins:

<https://youtube.com/shorts/puBdfPDEQmA?si=SkIKhgfkywgXbpVU>

<https://vm.tiktok.com/ZMA6uL4tX/>

Formatações print()

Vimos que a **saída de dados** é a interação do sistema com o usuário.

Vejam algumas formatações do print().

Para todos os exemplos, utilizaremos as variáveis abaixo com os seus respectivos conteúdos

Code

```
1 nome = "Edson de Oliveira"  
2 idade = 50  
3 altura = 1.71
```

Dica

O print() do Python por padrão exibe a informação e muda o cursor de linha.

FORMA CLÁSSICA

Esta é a forma que a maioria das linguagens de programação aceitam: separar as variáveis e/ou mensagens com vírgula:

Code

```
7 print(nome, idade, altura)
```

Na **linha 7** a vírgula determina um espaço depois de exibir a informação.

Output

```
Edson de Oliveira 50 1.71
```

FORMA CLÁSSICA 2

Alternando rótulos e variáveis.

Quando desejamos mesclar mensagens com variáveis podemos fazer desta forma:

Code

```
8 print("Nome: ", nome, "Idade: ", idade, "Altura: ", altura)
```

Na **linha 8** há uma mesclagem entre Rótulos e variáveis separadas com vírgula.

Output

```
Nome:  Edson de Oliveira Idade:  50 Altura:  1.71
```

Dica

A vírgula permite separar dados de tipos diferentes

FORMA CLÁSSICA 3

Utilizando o caractere de Scape ‘\n’.

Quando precisarmos quebrar a linha dentro de um print, usamos ‘\n’.

Code

```
9 print("Nome: ", nome, "\nIdade: ", idade, "\nAltura: ", altura)
```

Na **linha 9** houve o acréscimo do ‘\n’ antes dos da mudança de linha.

Este caractere de controle de fluxo força o desvio do cursor para a próxima linha.

Output

```
Nome: Edson de Oliveira
Idade: 50
Altura: 1.71
```

Dica

Pesquise outros caracteres de Scape.

UTILIZANDO CONCATENAÇÃO +

A concatenação também serve para mesclar informações e variáveis [mas somente] str().

Code

```
5 print("Nome: " + nome + "\nIdade: " + str(idade) + "\nAltura: " + str(altura))
```

Para funcionar foi feito o casting das variáveis não strings para str().

Output

```
Nome: Edson de Oliveira  
Idade: 50  
Altura: 1.71
```

Dica

Se usar a concatenação '+' com valores não str() ocasionará erro de compilação.

UTILIZANDO O FORMAT()

A função `format()` é uma das alternativas para não precisarmos mesclar variáveis e mensagens.

Em apenas uma string é possível exibir todas as alternâncias de mensagens e variáveis. As chaves `{}` determinam onde as variáveis serão exibidas.

Code

```
7 print("\nNome: {0}\nIdade: {1}\nAltura: {2}".format(nome, idade, altura))
```

Output

```
Nome: Edson de Oliveira  
Idade: 50  
Altura: 1.71
```

Dica

Os índices entre `{}` não precisam estar na ordem e podem ser repetidos no mesmo `print()`.

UTILIZANDO O FORMAT() 2

Ao invés de colocarmos os índices podemos criar um ‘aliás’ (apelido) para as variáveis.

Code

```
7 print("\nNome: {n}\nIdade: {i}\nAltura: {a}".format(n=nome, i=idade, a=altura))
```

O aliás não é uma variável. Ele funciona somente na linha da sua utilização.

Output

```
Nome: Edson de Oliveira  
Idade: 50  
Altura: 1.71
```

Dica

Os alias entre {} não precisam estar na ordem e podem ser repetidos no mesmo print().

UTILIZANDO O F'PRINT()

O 'f' print permite formatar variáveis e textos de uma forma mais simples.

Code

```
5 print(f"Nome: {nome}\nIdade: {idade}\nAltura: {altura}")
```

A facilidade está em colocar a variável diretamente entre chaves {}.

Output

```
Nome: Edson de Oliveira  
Idade: 50  
Altura: 1.71
```

Dica

Repare que para esta forma funcionar, devemos colocar o 'f' antes de abrir a string.

UTILIZANDO O TRIPLE QUOTES

Para facilitar a edição de textos podemos utilizar o triplo quotes ("""msg""") ao invés de vários prints().

Code

```
5 print(f"""  
6 Nome: {nome}  
7 Idade: {idade}  
8 Altura: {altura}  
9 """)
```

Output

```
Nome: Edson de Oliveira  
Idade: 50  
Altura: 1.71
```

Dica

Repare que neste caso não é necessário usar o '\n' deixando o código mais limpo. Podem ser utilizadas também as aspas simples 'msg'

UTILIZANDO O %

O % é uma herança de formatação da linguagem C. Com este recurso podemos aprimorar a formatação.

Code

```
5 print("Nome: %s \nIdade: %d\nAltura: %f" % (nome, idade, altura))
```

Output

```
Nome: Edson de Oliveira
Idade: 50
Altura: 1.71
```

Dica

*%s exibe str() | %d exibe int() [decimal] | %f exibe float()
Pesquise outros formatos do %*

PLUS

Seguem algumas formatações que funcionam quase todos os formatos vistos:

Code

```
1  pedido = 36
2  valor1 = 45.983
3  valor2 = 2394.4
4  valor3 = 998.88
5  valor_decimal = 12
6  print(f"""
7  Pedido.....: {pedido}
8  Pedido.....: {pedido:5d}
9  Pedido.....: {pedido:05d}
10
11 Valor 1.....: {valor1}
12 Valor 2.....: {valor2}
13 Valor 3.....: {valor3}
14
15 Valor 1.....: {valor1:10.2f}
16 Valor 2.....: {valor2:10.2f}
17 Valor 3.....: {valor3:10.2f}
18
19 Decimal: {valor_decimal} | Octal: {valor_decimal:o} | Hexadecimal: {valor_decimal:x}
20 """)
```

PLUS

Seguem algumas formatações que funcionam quase todos os formatos vistos:

Output

```
Pedido.....: 36  
Pedido.....: 36  
Pedido.....: 00036
```

```
Valor 1.....: 45.983  
Valor 2.....: 2394.4  
Valor 3.....: 998.88
```

```
Valor 1.....: 45.98  
Valor 2.....: 2394.40  
Valor 3.....: 998.88
```

```
Decimal: 12 | Octal: 14 | Hexadecimal: c
```

PLUS

Seguem algumas formatações que funcionam quase todos os formatos vistos:

Dica

Linha 8: A formatação `{pedido:5d}` significa exibir um valor `int()` em 5 bytes.

Linha 9: A formatação `{pedido:05d}` significa exibir um valor `int()` em 5 bytes completando com zeros à esquerda.

Linhas 15, 16 e 17: A formatação `{valor1:10.2f}` significa exibir um valor `float()` em 10 bytes com 2 casas decimais.

Linha 19: A formatação `{valor_decimal:o}` exibe em octal e `{valor_decimal:x}` exibe em Hexadecimal.

PESQUISEM OUTRAS FORMATAÇÕES

Exercícios:

Escreva os seguintes subalgoritmo e desenvolva-os em Python:

- 1) Dado um valor pelo usuário, calcular e exibir o seu dobro.
- 2) Dado um valor pelo usuário, calcular e exibir o seu cubo.
- 3) Dados quatro números pelo usuário, calcular e exibir a sua média.
- 4) Dados os valores de a, b e c, calcular o valor de delta.
- 5) Dada a temperatura em Célsius, calcular e exibir o equivalente em Fahrenheit.
- 6) Dada a temperatura em Fahrenheit, calcular e exibir o equivalente em Célsius.
- 7) Dado um valor, calcular e exibir o próximo múltiplo de 5.
- 8) Um caixa eletrônico dispensa cédulas de 10, 20, 50 e 100 reais. Dada uma quantia pelo usuário, informe quantas cédulas de cada é necessária para compor este valor.

Exercícios:

Escreva os seguintes subalgoritmo e desenvolva-os em Python:

1. Dado um valor informado pelo usuário, calcular e exibir o seu dobro.

Entrada: 10

Saída: O dobro de 10 é 20.

2. Dado um valor informado pelo usuário, calcular e exibir o seu cubo.

Entrada: 3

Saída: O cubo de 3 é 27.

3. Dados quatro números informados pelo usuário, calcular e exibir a sua média.

Entrada: 8, 6, 7, 9

Saída: A média é 7.5.

Exercícios:

Escreva os seguintes subalgoritmo e desenvolva-os em Python:

4. Dados os valores de a, b e c, calcular o valor de delta.

Fórmula: $\Delta = b^2 - 4ac$

Entrada: a = 1, b = 5, c = 6

Saída: O valor de delta é 1.

5. Dada a temperatura em Celsius, calcular e exibir o equivalente em Fahrenheit

(Fórmula: $F = (C \times 9/5) + 32$)

Entrada: C = 25

Saída: 25°C equivalem a 77.0°F.

Exercícios:

Escreva os seguintes subalgoritmo e desenvolva-os em Python:

6. Dada a temperatura em Fahrenheit, calcular e exibir o equivalente em Celsius. (Fórmula: $C = (F - 32) \times 5/9$)

Entrada: F = 98.6

Saída: 98.6°F equivalem a 37.0°C.

7. Dado um valor, calcular e exibir o próximo múltiplo de 5.

Entrada: 12

Saída: O próximo múltiplo de 5 após 12 é 15.

Exercícios:

Escreva os seguintes subalgoritmo e desenvolva-os em Python:

8. Um caixa eletrônico dispensa cédulas de 10, 20, 50 e 100 reais. Dada uma quantia pelo usuário, informar quantas cédulas de cada valor são necessárias para compor este montante.

Entrada: 380

Saída:

3 cédulas de 100

1 cédula de 50

1 cédula de 20

1 cédula de 10

BeabaProgramAcao

Prof. Edson de Oliveira



@BeabaProgramAcao



@BeabaDaProgramacaoEds